Assignment 2: BST and Red-Black Tree Benchmarking

Data Structures and Algorithms

Eyad Amr

April 6, 2026

Contents

1 System Specifications **2**

2 Methodology **2**
2.1 Data Structures 2
2.2 Input Distributions 2
2.3 Measurement Protocol 2

3 Tree Height After Insertion **3**

4 Benchmarking Results **3**
4.1 Fully Random Input 3
4.2 Nearly Sorted with 1% Misplaced Elements 4
4.3 Nearly Sorted with 5% Misplaced Elements 4
4.4 Nearly Sorted with 10% Misplaced Elements 5
4.5 RBTree Speedup over BST 5

5 Data Visualizations **6**

6 Analysis **7**
6.1 Effect of Input Distribution on BST 7
6.2 RBTree Consistency 7
6.3 Insert Operation 7
6.4 Contains Operation 7
6.5 Delete Operation 7
6.6 Tree Sort vs MergeSort 7
6.7 Conclusion 8

System Specifications

All benchmarks were conducted on the following hardware and software environment:

Component	Details
CPU	13th Gen Intel® Core™ i7-13700HX
RAM	15 GiB Total (11 GiB used during benchmarking)
OS	Ubuntu 24.04.4 LTS (Noble)
JDK	OpenJDK 21.0.10 (build 21.0.10+7-Ubuntu-124.04)
JVM	OpenJDK 64-Bit Server VM (mixed mode, sharing)
Build	Apache Maven, Java 21

Table 1: Benchmarking Environment

Methodology

Data Structures

Two tree-based data structures were implemented in Java:

- **Binary Search Tree (BST):** Standard recursive implementation with insert, delete, contains, inOrder, height, and size operations.
- **Red-Black Tree (RBTree):** Self-balancing BST with iterative insert and delete, maintaining all four Red-Black properties via rotations and recoloring.

Input Distributions

All benchmarks used arrays of $n = 100{,}000$ integers under four distributions:

- **Fully Random:** Integers drawn uniformly from $[0,\ 10n]$ using a fixed seed.
- **Nearly Sorted 1%:** Sorted sequence $0, 1, \ldots, n-1$ with 1,000 random swaps.
- **Nearly Sorted 5%:** Same base, with 5,000 random swaps.
- **Nearly Sorted 10%:** Same base, with 10,000 random swaps.

Measurement Protocol

- Each benchmark was run **5 times** and the mean, median, and standard deviation were reported.
- Both trees were evaluated on **identical inputs** using fixed seeds for reproducibility.
- All correctness validation code (`VALIDATE = false`) and debug logging (`logback` level `WARN`) were disabled before benchmarking.
- Timing was performed using `System.nanoTime()` and converted to milliseconds.

Tree Height After Insertion

The height of the tree after inserting all 100,000 elements reveals the structural difference between BST and RBTree:

Distribution	BST Height	RBTree Height
Fully Random	42	20
Nearly Sorted 1%	3145	30
Nearly Sorted 5%	1577	30
Nearly Sorted 10%	1010	29

Table 2: Tree height after inserting 100,000 elements

The BST degenerates to near-linear height on nearly-sorted inputs (e.g., height 3145 for 1% disorder), while the RBTree maintains a near-optimal logarithmic height (≈ 30) across all distributions.

Benchmarking Results

All times are in milliseconds (ms). Each entry reports Mean / Median / StdDev over 5 runs.

Fully Random Input

Structure	Operation	Mean (ms)	Median (ms)	StdDev (ms)
BST	Insert	64.05	64.84	16.48
	Contains	13.72	10.12	6.94
	Delete	15.87	17.47	4.32
	Sort	52.00	54.47	5.95
RBTree	Insert	51.93	45.83	14.00
	Contains	20.94	17.66	7.29
	Delete	8.63	8.60	2.53
	Sort	40.32	40.42	4.46
MergeSort	Sort	10.71	9.77	1.94

Table 3: Fully Random Input Benchmark Results

Nearly Sorted with 1% Misplaced Elements

Structure	Operation	Mean (ms)	Median (ms)	StdDev (ms)
BST	Insert	537.92	564.80	58.50
	Contains	171.35	159.08	26.06
	Delete	125.15	124.70	2.89
	Sort	488.84	494.39	14.26
RBTree	Insert	22.68	22.85	1.10
	Contains	17.19	16.40	3.41
	Delete	7.72	8.06	0.91
	Sort	28.01	26.32	3.47
MergeSort	Sort	8.62	8.60	0.03

Table 4: Nearly Sorted 1% Benchmark Results

Nearly Sorted with 5% Misplaced Elements

Structure	Operation	Mean (ms)	Median (ms)	StdDev (ms)
BST	Insert	112.01	108.04	7.32
	Contains	33.12	29.01	8.97
	Delete	40.20	41.33	9.04
	Sort	127.05	121.84	15.88
RBTree	Insert	25.92	25.98	0.37
	Contains	16.36	16.73	1.70
	Delete	6.79	8.03	2.11
	Sort	35.72	30.00	8.29
MergeSort	Sort	4.18	3.87	0.64

Table 5: Nearly Sorted 5% Benchmark Results

Nearly Sorted with 10% Misplaced Elements

Structure	Operation	Mean (ms)	Median (ms)	StdDev (ms)
BST	Insert	82.16	78.58	11.38
	Contains	25.05	21.02	7.73
	Delete	32.90	39.88	9.56
	Sort	80.56	84.09	5.80
RBTree	Insert	28.18	27.83	1.29
	Contains	19.91	21.42	5.40
	Delete	7.46	8.34	1.59
	Sort	31.55	32.23	0.98
MergeSort	Sort	4.50	4.48	0.09

Table 6: Nearly Sorted 10% Benchmark Results

RBTree Speedup over BST

The following table shows how many times faster RBTree is compared to BST (ratio > 1 means RBTree is faster):

Operation	Random	NS-1%	NS-5%	NS-10%
Insert	1.23×	23.72×	4.32×	2.92×
Contains	0.66×	9.97×	2.03×	1.26×
Delete	1.84×	16.21×	5.92×	4.41×
Sort	1.29×	17.46×	3.56×	2.55×

Table 7: RBTree speedup factor over BST (Mean time BST / Mean time RBTree)

Data Visualizations

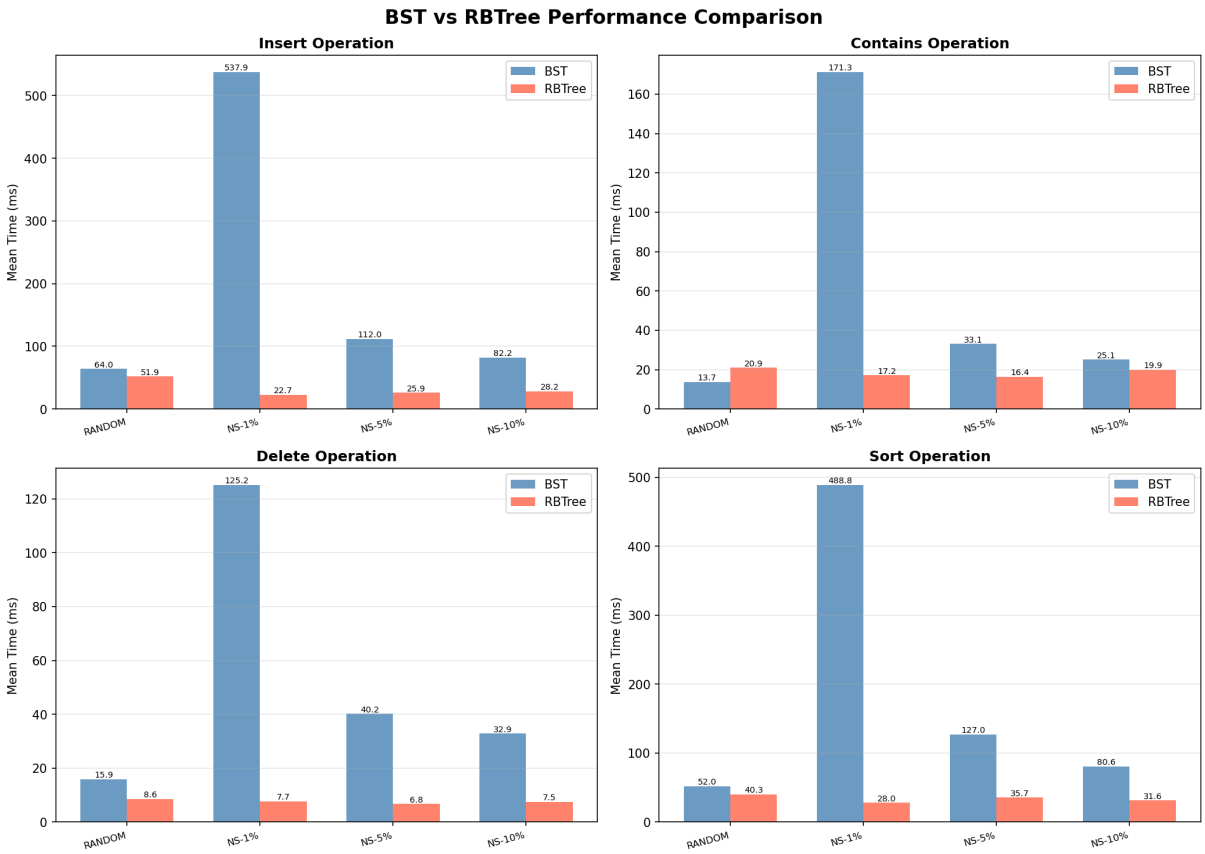

Figure 1: BST vs RBTree performance comparison across all operations and input distributions

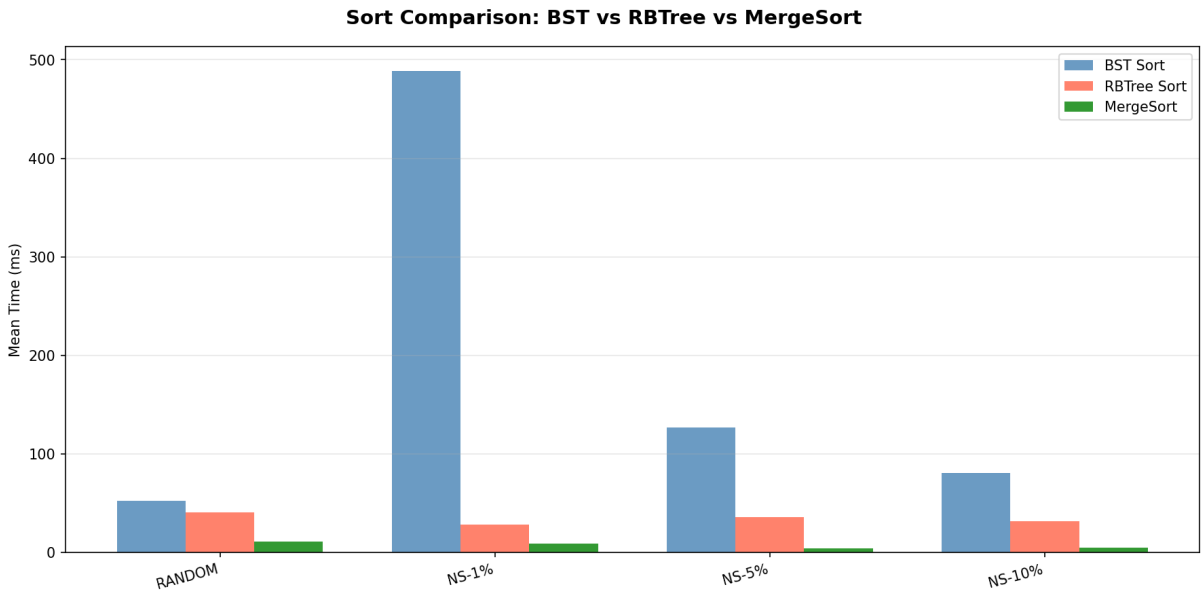

Figure 2: Sort comparison: BST Tree Sort vs RBTree Tree Sort vs MergeSort

Analysis

Effect of Input Distribution on BST

The BST is highly sensitive to input order. On fully random input, the BST achieves a height of 42, close to the theoretical average of $O(\log n)$. However, as the input becomes more sorted, the BST degenerates. With only 1% disorder, the BST height jumps to 3,145 — nearly 75× the RBTree height of 30. This degeneration directly causes all operations to slow down dramatically, since every traversal follows a near-linear path through the tree.

As disorder increases from 1% to 10%, BST performance improves substantially. The height drops from 3,145 to 1,010 to 1,577 respectively, and operation times decrease accordingly. This confirms that even a small amount of randomness in the input significantly improves BST performance.

RBTree Consistency

The RBTree maintains a height between 20 and 30 across all four distributions. This consistency is the direct result of its self-balancing mechanism — rotations and recoloring after every insert and delete ensure the black-height property is never violated. The near-constant height guarantees $O(\log n)$ performance regardless of input order.

Insert Operation

On random input, BST insert (64.05ms) is slightly slower than RBTree (51.93ms), giving RBTree a 1.23× speedup. On nearly-sorted input the difference becomes extreme: BST takes 537.92ms vs RBTree's 22.68ms on NS-1%, a 23.72× speedup. This is because BST must traverse thousands of nodes to find insertion positions in a degenerate tree, while RBTree's balanced structure keeps traversal depth at ≈ 30.

Contains Operation

On random input, BST contains (13.72ms) is actually faster than RBTree (20.94ms). This is because the RBTree insert overhead adds rotation and recoloring overhead that slightly affects cache locality. However on nearly-sorted input, RBTree is 9.97× faster than BST on NS-1%.

Delete Operation

RBTree is consistently faster at deletion across all distributions. On random input the speedup is 1.84×, growing to 16.21× on NS-1%. The RBTree delete implementation uses an iterative approach with parent pointers, which also benefits from better cache behavior.

Tree Sort vs MergeSort

Both tree-based sorts are significantly slower than MergeSort across all distributions. MergeSort completes in 3–11ms while even RBTree sort takes 28–40ms. The gap is due to:

- **Cache efficiency:** MergeSort operates on contiguous arrays; trees use pointer-based structures with poor cache locality.

- **Overhead:** Tree sort requires building the tree (with all its structural maintenance) before traversal, while MergeSort sorts in-place.
- **Memory allocation:** Each tree node requires separate heap allocation, adding significant GC pressure.

On nearly-sorted input, BST tree sort collapses to 488.84ms (NS-1%) due to tree degeneration, making it 57× slower than MergeSort. RBTree sort remains stable at ≈28–36ms, only 3–8× slower than MergeSort.

Conclusion

The benchmarks clearly demonstrate that the Red-Black Tree is the superior choice for workloads where input order cannot be guaranteed. Its $O(\log n)$ guarantee across all distributions makes it robust against adversarial or nearly-sorted inputs that cripple plain BSTs. However, for pure sorting tasks, dedicated comparison-based algorithms like MergeSort outperform tree-based approaches by an order of magnitude due to better cache locality and lower constant factors.

On fully random input, the BST is competitive and occasionally faster (notably for *contains*), suggesting that BSTs remain viable when input distribution is known to be random. The choice between BST and RBTree should therefore depend on the expected input characteristics of the application.